

Estimations of Quantitative Features for SignBase Objects

Description

This file contains the code for estimating the feature values (TTR, unigram entropy, entropy rate, repetition rate) per sign sequence on paleolithic objects from the Swabian Aurignacian. These features were tested on a wider sample of linguistic and non-linguistic sign sequences in Bentz (2023). The repetition measure was tested also in Sproat (2014).

Session Info

Give the session info (reduced).

```
## [1] "R version 4.4.0 (2024-04-24)"  
## [1] "x86_64-pc-linux-gnu"
```

Load libraries

If the libraries are not installed yet, you need to install them using, for example, the command: `install.packages("ggplot2")`. For the Hrate package this is different, since it comes from github. The devtools library needs to be installed, and then the `install_github()` function is used.

```
library(stringr)  
library(entropy)  
library(gsubfn)
```

```
## Loading required package: proto
```

```
#library(devtools)  
#install_github("dimalik/Hrate")  
library(Hrate)
```

Give the package versions.

```
##   Hrate  gsubfn  proto entropy stringr  
##   "0.2"   "0.7"  "1.0.0" "1.3.1" "1.5.1"
```

External functions

```
# repetition rate function rrate()  
source("~/Github/PaleoSigns/code/functions/repRate_function.R")
```

Load files

Load the files with the cleaned and randomized sequences.

```
# signBase data
sign.data <- read.csv("~/Github/PaleoSigns/data/signBase/signBase_randomized.csv")
```

SignBase (Swabian Aurignacian) Estimations

Estimations of quantitative features for the objects and their sign codings as found in signBase.

```
# set counter
counter = 0
# initialize dataframe to append results to
estimations.df <- data.frame(object_id = character(0),
                             num.tokens = numeric(0),
                             num.types = numeric(0),
                             huni.chars = numeric(0),
                             hrate.chars = numeric(0),
                             hrate.chars.random = numeric(0),
                             ttr.chars = numeric(0),
                             rm.chars = numeric(0))

# start time
start_time <- Sys.time()
for (i in 1:nrow(sign.data)){
  try({ # if the processing fails for a certain row, there will be no output for this row,
    # but the try() function allows the loop to keep running

    # collect info from row
    object_id <- sign.data$object_id[i]
    coding.sample <- as.character(sign.data$coding_clean[i])
    coding.sample.random <- as.character(sign.data$coding_random[i])

    # split sequences per object into UTF-8 characters
    chars <- unlist(strsplit(coding.sample, ""))
    chars.random <- unlist(strsplit(coding.sample.random, ""))

    # get number of sign tokens and types
    num.tokens <- length(chars)
    num.types <- length(unique(chars))

    # preprocess text with bag-of-words normalization
    # note: this is an important step! Without it, entropy rate estimation will fail
    # to yield realistic results due to issues with processing special characters.
    chars <- PreprocessText(chars)
    chars.random <- PreprocessText(chars.random)

    # unigram entropy estimation
    # calculate unigram entropy for characters (note that this remains the same
    # for random ordering of character strings)
    chars.df <- as.data.frame(table(chars))
    # print(chars.df)
```

```

huni.chars <- entropy(chars.df$Freq, method = "ML", unit = "log2")

# entropy rate estimation
# the values chosen for max.length and every.word will crucially
# impact processing time. In case of "max.length = NULL" the length is n units
# here use the try() function since entropy rate estimation can fail when particular
# special characters are in the strings
hrate.chars <- try(get.estimate(text = chars, every.word = 1, max.length = NULL))
if (class(hrate.chars) == "numeric") {
  hrate.chars <- hrate.chars
} else {
  hrate.chars = "NA"
}

# for random ordering of characters strings
hrate.chars.random <- try(get.estimate(text = chars.random, every.word = 1, max.length = NULL))
if (class(hrate.chars.random) == "numeric") {
  hrate.chars.random <- hrate.chars.random
} else {
  hrate.chars.random = "NA"
}

# calculate type-token ratio (ttr)
# note that this is the same for random ordering of strings
ttr.chars <- nrow(chars.df)/sum(chars.df$Freq)

# calculate repetition rate
rm.chars <- rrate(chars, method = "lsmo")

# append results to dataframe
local.df <- data.frame(object_id, num.tokens, num.types, huni.chars, hrate.chars,
  hrate.chars.random, ttr.chars, rm.chars)
estimations.df <- rbind(estimations.df, local.df)
}) # end of try()
# counter
counter <- counter + 1
# print(counter)
}
end_time <- Sys.time()
end_time - start_time

```

Time difference of 1.544752 secs

#estimations.df

Merge estimations with original file.

```
sign.data.est <- merge(sign.data, estimations.df, by = "object_id")
```

Give head of estimation data frame (selected columns) for visual checking of the output data structure.

```
head(sign.data.est[c("object_id", "coding_clean", "num.tokens", "num.types", "huni.chars",
  "hrate.chars", "ttr.chars", "rm.chars")])
```

```
##   object_id      coding_clean num.tokens num.types huni.chars hrate.chars
## 1  bst0001 vvvvvvvvvvvvvvvvvv      18         1  0.0000000  0.6171397
## 2   gkl0001      ||| //////////////_V      12         5  1.8962406  1.5295563
```

```
## 3   gkl0002          vvvvv      5      1 0.0000000 0.5711903
## 4   gkl0003          vvvv vvvv   9      2 0.5032583 0.9060047
## 5   gkl0004      vvvvvvvvvvvvvv 14      1 0.0000000 0.6379550
## 6   gkl0005          |||        3      1 0.0000000 0.4308271
##      ttr.chars  rm.chars
## 1 0.05555556 1.0000000
## 2 0.41666667 0.6363636
## 3 0.20000000 1.0000000
## 4 0.22222222 0.7500000
## 5 0.07142857 1.0000000
## 6 0.33333333 1.0000000
```

Safe outputs to file.

```
write.csv(sign.data.est, "~/Github/PaleoSigns/results/features/signBase_features.csv", row.names = F)
```

Package References

```
## Bentz C, Alikaniotis D (2017). _Hrate: Implements LZ estimator for
## texts_. R package version 0.2, commit
## b67b4bb5f5430ce42d2844da30ddf5642540aa30,
## <https://github.com/dimalik/Hrate>.

## Wickham H (2023). _stringr: Simple, Consistent Wrappers for Common
## String Operations_. R package version 1.5.1,
## <https://CRAN.R-project.org/package=stringr>.

## Grothendieck G (2018). _gsubfn: Utilities for Strings and Function
## Arguments_. R package version 0.7,
## <https://CRAN.R-project.org/package=gsubfn>.

## Hausser J, Strimmer K (2021). _entropy: Estimation of Entropy, Mutual
## Information and Related Quantities_. R package version 1.3.1,
## <https://CRAN.R-project.org/package=entropy>.
```

Literature References

Bentz, C. (2023). The Zipfian Challenge: Learning the statistical fingerprint of natural languages. In Proceedings of the 27th Conference on Computational Natural Language Learning (CoNLL), pp. 27-37.

Sproat, R. (2014). A statistical comparison of written language and nonlinguistic symbol systems. *Language*, 457-481.